Лабораторна робота № 12

РОБОТА З ФАЙЛАМИ.

Мета роботи:

Вивчити поняття потоків і файлів для вводу/виводу даних.

Обладнання та програмне забезпечення:

– IBM сумісна персональна обчислювальна машина;

– онлайн компілятор мови програмування Сі, доступний за посиланням https://repl.it/languages/c

Завдання до роботи:

Написати програми на мові програмування Сі, котрі забезпечують обмін даними між периферійними пристроями (консоллю, файлами) і програмами.

Теоретичні відомості

В Сі файли – це логічна концепція, застосовна до всього, починаючи від дискових файлів і закінчуючи терміналами.

Файли Сі підтримують різний режим доступу. Може бути послідовний доступ або довільний доступ. Файл на жорсткому диску підтримує довільний доступ, а модем тільки послідовний.

Основні функції для роботи з файлами наведені в наступній таблиці:

Фунція	Опис
fopen()	Відкриває файл
fclose()	Закриває файл
fputc()	Записує символ в файл
fputs()	Записує стрічку у файл
fgetc()	Читає символ з файлу
fgets()	Читає стрічку з файлу
fseek()	Встановлює вказівник поточної позиції на вказаний байт файлу
ftell()	Повертає поточне значення вказівника у файлі
fprintf()	Для файлу те саме, що printf() для консолі
fscanf()	Для файлу те саме, що scanf() для консолі
feof()	Повертає істину, якщо досягнуто кінець файлу
ferror()	Повертає істину, якщо виникла помилка
rewind()	Скидає вказівник поточної позиції на початок файлу
remove()	Стирає файл
fflush()	Дозапис потоку у файл, очищення буферу файлу

В будь-яку програму, яка використовує роботу з файлами, повинен включатися **заголовковий файл `stdio.h`** . Він надає прототипи для функцій вводу/виводу і визначає три типи: *`size_t`*, *`fpos_t`* і *`FILE`*. Типи *`size_t`* і *`fpos_t`* – це беззнакові цілі числа.

Вказівник на файл – це вказівник на інформацію, яка визначає різні параметри файла, включаючи його ім'я, стан і поточну позицію. В принципі, вказівник на файл ідентифікує конкретний дисковий файл і використовується потоком для виконання операцій вводу/виводу. Вказівник на файл – це змінна-вказівник типу `FILE`. Щоб прочитати або записати файли, програмі треба використовувати вказівники на файли. Для створення файлової змінної-вказівника використовується оператор типу `FILE *fp;`

Потік зв'язується з конкретним файлом за допомогою **операції відкриття**. Функція `fopen()` відкриває для використання потік, зв'язує файл з даним потоком і потім повертає вказівник `FILE` на даний потік. Найчастіше файлом є дисковий файл. Функція `fopen()` має наступний прототип:

```
FILE *fopen(const char *ім'я_файлу, const char *режим);
```

де вказівник `режим` вказує на стрічку, яка містить бажаний режим відкриття файлу. `ім'я файлу` має бути стрічкою символів, яка надає коректне ім'я файлу операційній системі, і може містити вказання шляху. Допустимі значення для `режиму` показані в таблиці.

Режим	Призначення
r	Відкрити текстовий файл для читання. Файл повинен існувати.
w	Відкрити текстовий файл для запису. Якщо файлу не існує, він буде створений, інакше його вміст буде втрачено.
a	Запис в кінець текстового файлу. Якщо файлу не існує, то він буде створений. Операції позиціонування ігноруються.
rb	Відкрити двійковий файл для читання. Файл повинен існувати.
wb	Відкрити двійковий файл для запису. Якщо файлу не існує, він буде створений, інакше його вміст буде втрачено.
ab	Запис в кінець двійкового файла. Якщо файлу не існує, то він буде створений. Операції позиціонування ігноруються.
r+	Відкрити текстовий файл для читання/запису. Файл повинен існувати і не очищується при відкритті.
w+	Відкрити двійковий файл для запису. Якщо файлу не існує, він буде створений, інакше його вміст буде втрачено.
a+	Запис в кінець та оновлення текстового файлу. Якщо файлу не існує, то він буде створений. Операції позиціонування працюють лише для читання, для запису ігноруються. Можна як читати, так і писати.
r+b	Відкрити двійковий файл для читання/запису. Файл повинен існувати і не очищується при відкритті.
w+b	Відкрити двійковий файл для читання/запису. Якщо файл не існує, він створюється, інакше вміст втрачається. Можна як читати, так і писати.
a+b	Запис в кінець двійкового файла або створення його для читання/запису. Операції позиціонування працюють лише для читання, для запису ігноруються.

Зазвичай для відкриття файлу застосовується наступна конструкція:

```
FILE *fp = fopen("test", "w");
if (fp == NULL) {
  puts("Cannot open file.");
  exit(1);
}
```

Цей метод дозволяє виявити помилки при відкритті файлу, наприклад, наявність захисту від запису або відсутність вільного місця на диску.

Якщо файл відкритий, може здійснюватися обмін між файлом і програмою.

Зазвичай для обміну використовується певна фіксована кількість даних.

Тому для вводу/виводу використовується буфер.

Зв'язок потоку з файлом знищується за допомогою операції закриття.

Закриття потоку викликає примусове скидання всього вмісту буфера у зовнішній пристрій (якщо необхідно, інформація буде доповнена для створення повного комплекту даних). Даний процес називається очищенням буфера, і він гарантує, що в буфері не залишиться інформації.

Всі файли закриваються автоматично, коли програма нормальним чином завершує роботу, виконавши функцію `main()`, або за допомогою виклику оператора `exit()`.

Проте, краще самому закрити файли, використовуючи `fclose()` в той момент, коли файл вже не потрібен, оскільки програма може завершитися неочікуваним чином, наприклад може зникнути живлення або виникнути помилка.

Запис в файл:

```
#include <stdio.h>

int main () {
  FILE * pFile;
  int n;
  char name [100];

  pFile = fopen ("myfile.txt","w");
  for (n=0 ; n<3 ; n++) {
    puts ("please, enter a name: ");
    scanf ("%s", name);
    fprintf (pFile, "Name %d [%-10.10s]n \n",n,name);
  }
  fclose (pFile);
  return 0;
}
```

Зчитування з файлу:

```
#include <stdio.h>

int main () {
  FILE * pFile;
  int n;
  char name [100];

  pFile = fopen ("myfile.txt","r");
  for (n=0 ; !feof(pFile) ; n++) {
    fscanf (pFile,"%s", name);
    printf ("String number %d= %s \n",n,name);
  }
  fclose (pFile);
  return 0;
}
```

Файлова система ANSI C надає дві функції, які дозволяють читати і писати блоки даних – `fread()` і `fwrite()`. Вони мають наступні прототипи:

```
size_t fread(void *буфер, size_t число_байт size_t об’єм, FILE *fp);

size_t fwrite(const void *буфер, size_t число_байт, size_t об’єм, FILE *fp);
```

У випадку `fread()`, `буфер` – це вказівник на область пам'яті, яка отримує дані із файла. У випадку `fwrite()`, `буфер` – це вказівник на інформацію, яка записується у файл. Довжина кожного елемента в байтах визначається в параметрі `число_байт`. Аргумент `об'єм` визначає, скільки елементів (кожен довжиною `число_байт`) буде прочитано або записано. Врешті, `fp` – це файловий вказівник на раніше відкритий потік.

Функція `fread()` повертає число прочитаних елементів. Це значення може бути меншим ніж об'єм, якщо був досягнутий кінець файлу або сталася помилка. Функція `fwrite()` повертає число записаних елементів. Це значення рівне параметру `об'єм`, якщо не виникла помилка.

Одне із найбільш корисних застосувань `fread()` і `fwrite()` – це читання та запис блоків даних типу масивів або структур.

Для **видалення файлів** використовується функція `remove()`. Вона має наступний прототип:

```
int remove(const char *имя_файла);
```

У випадку успішного виконання вона повертає нуль, а у випадку невдалого – значення, відмінне від нуля.

Функція `ferror()` використовується для перевірки, чи виконана операція не привела часом до помилки. Функція ferror() має наступний прототип:

```
int ferror(FILE *fp);
```

де `fp` – це допустимий вказівник на файл. Функція `ferror()` повертає істину, якщо в результаті виконання останньої операції сталася помилка. В іншому випадку вона повертає хибне. Оскільки кожна файлова операція змінює стан помилки, `ferror()` має викликатися після кожної файлової операції, інакше помилка може загубитися.

Консольний ввід/вивід відноситься до подій, що виникають від клавіатури або на екрані. Для обробки цих подій використовується бібліотека stdio.h

```
#include <stdio.h>
#include <ctype.h> // Файл <ctype.h> містить прототипи для десятка функцій
для класифікації символів.
int main(void) {
  char ch;
  do {
    ch = getchar();
    if(islower(ch)) putchar(toupper(ch)); // islower(ch)) putchar(toupper(ch)
належать бібліотеці ctype.h
    else putchar (tolower(ch));
  } while ( ch != '.'); // використання . для кінця роботи
  return 0;
}
```

ПОРЯДОК ВИКОНАННЯ РОБОТИ:

1. Опрацювати і засвоїти матеріал наведений в теоретичних відомостях.
2. Написати програму, яка реалізує ввід з консолі 10+ довільних стрічок і записує їх у створений програмою файл «mfile.txt».
3. Написати програму, котра зчитує файл «mfile.txt» (записаний в п.2), виводить його вміст на дисплей, підраховує та виводить на екран кількість стрічок, а також середню кількість символів у стрічці. Після цього програма записує вміст файлу «mfile.txt» в оберненому порядку (від останньої стрічки до першої, від останнього символа у стрічці до першого) у файл «xfile.txt».
4. Написати програму, яка з використанням блоків даних (функції `fread()` та `fwrite()`) копіює файли типу “*.pdf” та “*.jpg”
5. В звіті навести копії екранів, написаний код, та вміст файлів.
6. Зробити висновки.